CACHE SIMULATOR IN RUST

Prepared by Nicholas Catani

University of London
MS in Computer Science

CSM030 – Computer Systems

INDEX

1. Introduction
2. Scope and Interpretation of the Specs.
3. Assumptions
4. Software Architecture and Data Structures
5. Address Decomposition
6. Replacement Policy Implementation
7. Trace Processing
8. Testing and Verification
9. Conceptual Extension A
10. Conceptual Extension B
11. Conclusion

1. Introduction

This project implements a simplified cache simulator in Rust. The purpose of the simulator is to analyze how cache memories affect program performance by tracking cache hits, misses, and evictions during memory accesses.

Modern processors rely heavily on caching mechanisms to reduce memory access latency. Instead of always retrieving data from RAM, frequently used data is stored in smaller and faster cache memories. When the CPU requests data, it first checks the cache. If the requested data is present, a cache hit occurs. By contrast, a cache miss occurs, and the data must be fetched from main memory.

The simulator developed in this project models cache behavior using configurable parameters and processes memory access traces generated from real programs. The final output reports the number of hits, misses, and evictions produced during simulation.

2. Scope and Interpretation of the Specification

The assignment specification describes a simplified cache model derived from real multi-level caches. In practice, modern processors include multiple levels of cache (L1, L2, L3). However, for this simulator all cache levels are treated as a single logical cache.

The simulator accepts three cache parameters at runtime:
- s: number of sets index bits
- E: number of cache lines per set
- b: number of blocks offset bits

These parameters determine the cache structure:
- Number of sets = 2^s
- Block size = 2^b bytes
- Each set contains E lines

The simulator reads a memory access trace file where each line represents an operation:

OPERATION	MEANING
I	Instruction load (ignored)
L	Data load
S	Data store
M	Data modify (load followed by store)

Only data cache operations (L, S, M) are simulated.

The simulator must produce exactly one line of output:
hits:X misses:Y evictions:Z

This strict output format allows automatic testing against the reference simulator.

3. Assumptions

Several simplifying assumptions were made based on the specification:

1. Instruction loads are ignored because the assignment focuses only on data cache behavior.
2. Memory accesses do not cross block boundaries; thus, the request size in the trace file can be ignored.
3. Block offset bits are not required for simulation because blocks are fetched entirely.
4. The simulator assumes 64-bit memory addresses.
5. Cache parameters provided at runtime are assumed to be valid within the specified constraints.

These assumptions allow the simulator to focus purely on cache lookup behavior.

4. Software Architecture and Data Structures

The program is organized into several logical components:

COMPONENT	RESPONSIBILITY
Cache	Represents the simulated cache structure.
Line	Represents a single cache line.
simulate()	Processes the trace file.
parse_args()	Parses runtime parameters.
main()	Coordinates execution.

This modular structure improves readability and simplifies testing.

CACHE REPRESENTATION

The cache is implemented using a two-dimensional vector:
Vec<Vec<Line>>

- Outer vector -> cache sets
- Inner vector -> cache lines within each set

This structure satisfies the requirement that the cache be dynamically allocated at runtime, allowing the simulator to support arbitrary values of s and E.

Each cache line stores the following metadata:

FIELD	PURPOSE
valid	Indicates whether the line contains valid data.
tag	Identifies the memory block stored in the line.
last_used	Timestamp used to implement LRU replacement.

5. Address Decomposition

Each memory address is divided into three components:
[tag | set index | block offset]

The simulator extracts these values using bit operations.

SET INDEX

The set index determines which cache set should be accessed:
set_index = (address >> b) & ((1 << s) - 1)

- “>> b” removes block offset bits
- masking isolates the s set bits

TAG

The tag identifies the memory block within a set:
tag = address >> (s + b)

The tag is compared with existing cache lines to determine whether a hit occurs.

**In this simulator, only the set index and tag fields are required. The block offset is not used because the simulator does not model individual bytes within a block. Instead, it only tracks whether the entire block is present in the cache.

6. Replacement Policy Implementation

The simulator implements the Least Recently Used replacement policy. LRU evicts the cache line that has not been accessed for the longest period of time. To implement LRU, the simulator maintains a global logical timestamp:
time

Each time a memory access occurs, the timestamp increments. The accessed line records this timestamp in its “last_used” field. When a cache set becomes full and a new block must be inserted, the simulator selects the line with the smallest “last_used” value, indicating it was accessed least recently. This approach provides a straightforward and efficient LRU implementation.

7. Trace Processing

The simulator processes the trace file sequentially.

Each line of the trace file contains:
[space] operation address, size

Instruction loads (I) are ignored.

For each data operation:
- L or S -> one cache access.
- M -> two accesses (load followed by store)

This behavior matches the specifications.

8. Testing and Verification

Unit tests were implemented to verify the correctness of core cache behaviors. These tests ensure that the simulator correctly updates hit, miss, and eviction counts under controlled scenarios. All simple tests passed successfully, providing confidence in the correctness of the core cache simulator logic.

1. **Cold_miss_then_hit**

This test verifies the basic behavior of a cache when a memory address is accessed for the first time. When a block is accessed for the first time, the cache cannot contain it yet, thus, a cold miss. The block is then inserted into the cache. Then, second access to the same address should result in a cache hit.

2. **Direct_mapped_conflict_causes_eviction**

This test verifies the behavior of a direct-mapped cache (E = 1) when two different blocks map to the same set. When two memory addresses map the same cache set in a direct-mapped cache, the second block must replace the first one.

3. **Two_way_set_fills_before_eviction**

This test verifies correct behavior in a set-associative cache where multiple lines exist in each set. A set-associative cache should fill all available lines in a set before performing an eviction.

4. **Lru_replaces_least_recently_used_line**

This test verifies the implementation of the LRU replacement policy. When eviction is required, the simulator should remove the block that was least recently accessed.

5. **Modify_is_two_accesses**

This test verifies that the simulator correctly handles modifying operations (M). According to the specs, a modify operation represents (load + store). Thus, it should be treated as a two cache accesses.

9. Conceptual Extension A: Configurable Replacement Policies

The current implementation supports only the LRU replacement policy. However, the simulator could be extended to support additional policies such as:
- FIFO
- Random replacement

One possible design approach would be to abstract the replacement policy into a separate component. In Rust, this could be implemented using either:
- an enum representing policy types, or
- a trait defining replacement behavior

The cache structure could then call the appropriate policy when an eviction is required.

```
Trait ReplacementPolicy {
        fn select_victim(set: &[Line]) -> usize;
}
```

At runtime, the desired policy could be selected through an additional command-line parameter. This design would allow new policies to be added without modifying the core cache simulation logic.

10. Conceptual Extension B: Multi-level Cache Simulation

Real processors use multiple levels of cache (L1, L2, L3). The simulator could be extended to model a two-level cache hierarchy.

In such a design:
1. The CPU first checks L1 cache.
2. If L1 misses, the request is forwarded to L2.
3. If L2 misses, data is retrieved from main memory.

Two common hierarchy designs exist:

INCLUSIVE CACHE

All blocks stores in L1 must also exist in L2.
Evicting a block from L2 requires invalidating it in L1.

EXCLUSIVE CACHE

Blocks exist in only one level at a time.
Blocks may move between L1 and L2 as accesses occur.

Implementing a multi-level cache would require:
- separate cache structures for each level.
- coordination between levels during misses.
- additional metadata to maintain hierarchy consistency.

This extension would significantly increase the simulator's realism.

11. Conclusion

This project successfully implemented a configurable cache simulator in Rust. The simulator dynamically constructs cache structures based on runtime parameters and processes real memory access traces to determine cache performance metrics.

The use of modular design and dynamic data structures ensures the simulator remains flexible and extensible. Testing against the provided reference implementation confirmed the correctness of the simulation.

Potential future improvements include support for configurable replacement policies and multi-level cache hierarchies, which would bring the simulator closer to modeling real processor architectures.